

Tech Challenge – Fase 2

Julio Cesar Perroni - RM370079

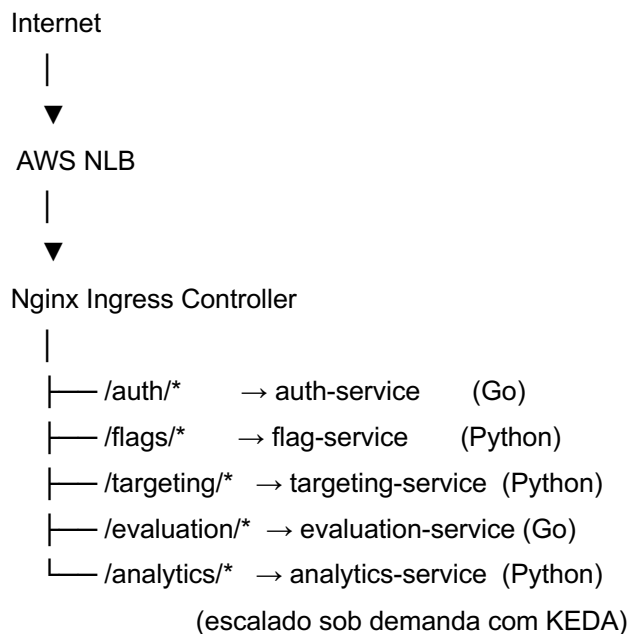
Discord: jc.dev86

Plataforma de Feature Flags — Visão Geral

Esta solução consiste em uma plataforma distribuída de feature flags, composta por cinco microserviços containerizados, orquestrados no Amazon EKS, utilizando Nginx Ingress Controller, KEDA para autoescalabilidade e três tecnologias de armazenamento de dados, cada uma escolhida conforme seu propósito específico.

Arquitetura da Solução

A plataforma é exposta externamente por meio de um balanceador de carga (AWS NLB), que direciona o tráfego ao Nginx Ingress Controller, responsável pelo roteamento para os microserviços:



Microserviços da Plataforma

auth-service — Autenticação

Responsável pela emissão e validação de API Keys, garantindo a segurança de todas as requisições da plataforma.

- Tecnologia: Go
- Justificativa: Baixa latência e alta eficiência em operações críticas (hash e validação)
- Armazenamento: RDS PostgreSQL (api_keys)
- Observação: Atua como ponto central de autenticação para todos os serviços

flag-service — Gerenciamento de Flags

Gerencia o ciclo de vida completo das feature flags.

- Tecnologia: Python (Flask)
- Responsabilidade: CRUD de flags (nome, descrição e estado global)
- Autenticação: Delegada ao auth-service
- Armazenamento: RDS PostgreSQL (flags)

targeting-service — Segmentação

Define regras de ativação das flags com base em critérios dinâmicos.

- Tecnologia: Python (Flask)
- Responsabilidade: Associação de regras de segmentação às flags
- Formato: Regras flexíveis em JSON (percentual, lista de usuários, critérios customizados)
- Autenticação: Delegada ao auth-service
- Armazenamento: RDS PostgreSQL (targeting_rules com JSONB)

evaluation-service — Motor de Avaliação

Serviço central da plataforma, responsável por determinar se uma flag está ativa para um usuário.

- Tecnologia: Go
- Responsabilidade: Avaliação de flags com base em user_id e flag_name
- Otimização: Cache em Redis por 30 segundos
- Eventos: Publicação assíncrona de avaliações em fila SQS
- Armazenamento:
 - Redis (cache)
 - SQS (eventos)

Diferencial: Projetado para alta performance, sendo o serviço mais requisitado da plataforma.

analytics-service — Processamento de Eventos

Responsável pelo processamento assíncrono dos eventos gerados.

- Tecnologia: Python
- Modelo: Worker (não expõe API REST)
- Responsabilidade: Consumo da fila SQS e persistência dos eventos
- Escalabilidade: Gerenciada via KEDA (escala até zero)
- Armazenamento: DynamoDB

Estratégia de Escalabilidade: KEDA vs HPA

A escolha do KEDA foi motivada pela necessidade de escalabilidade orientada a eventos, diferente do modelo tradicional baseado em infraestrutura.

Estratégia de Armazenamento

A arquitetura utiliza três soluções distintas de armazenamento, cada uma alinhada ao tipo de carga de trabalho.

RDS PostgreSQL — Dados Transacionais

Utilizado para entidades com ciclo de vida estruturado.

Motivações:

- Modelo relacional com consistência ACID
- Estrutura de dados bem definida
- Isolamento por serviço (resiliência)

Redis (ElastiCache) — Cache de Alta Performance

Utilizado pelo evaluation-service para reduzir latência.

Benefícios:

- Respostas em milissegundos
- Redução de chamadas entre serviços
- Tolerância a falhas (cache descartável)

DynamoDB — Armazenamento de Eventos

Projetado para ingestão massiva de dados.

Motivações:

- Escalabilidade automática
- Modelo append-only
- Alta taxa de escrita sem necessidade de gerenciamento de infraestrutura

Fluxo de Avaliação de Feature Flag

O processo de avaliação segue um fluxo otimizado para desempenho e desacoplamento:

1. Cliente → /evaluation/evaluate
2. Validação do token (auth-service)
3. Consulta ao cache (Redis)
 - └─ HIT → resposta imediata
 - └─ MISS → consulta flag-service + targeting-service
4. Execução das regras de segmentação
5. Retorno do resultado ao cliente
6. Envio assíncrono do evento → SQS → analytics-service → DynamoDB

Link do vídeo:

<https://drive.google.com/file/d/1aR7e4niJP9nAQAcJ4NPI1o6cyEgez2M6/view?usp=sharing>

Link do repositório:

<https://github.com/jc-perroni/fiap-fase-2>